# CBF Implementation Plan

## CBF Implementation Plan (Latent Space Planning)

### 1. Core Design Constraints

- **Safety convention**:
  - $B(z, o) \geq 0 \rightarrow$ Safe
  - $B(z, o) < 0 \rightarrow$ Unsafe
- **Nominal planner definition**:
  - Use **Goal + Prior losses only**
  - Remove collision classifier completely
- **Code constraint**:
  - Do **not modify any existing `.py` files**
  - Implement using **standalone scripts only**

---

### 2. Baseline Freezing (Reproducibility First)

Fix all sources of variation:

- Frozen **VAE checkpoint**
- Fixed **obstacle normalization**
- Fixed **random seeds**
- Fixed **evaluation scene set**

**Purpose**: Ensure fair **before vs after (CBF)** comparison

---

### 3. Dataset Construction

#### 3.1 State-Label Dataset

- Build latent dataset:

$$(z, o, y)$$

  where:
  - $z$: encoded state

  - $o$: obstacle representation
  - $y \in \{\text{safe}, \text{unsafe}\}$
- Source:
  - Existing collision dataset
  - Encode using frozen VAE

---

### 3.2 Transition-Pair Dataset (Critical)

- Collect:

$$(z_k, z_{k+1}^{nom}, o)$$

- Generation:
  - Run planner with **Goal + Prior only**
  - Log every optimization step
- Role:
  - Enables learning of **CBF dynamics constraint**

---

### 3.3 Dataset Splitting

- Split into:
  - Train / Validation / Test
- Ensure:
  - **Scenario-level separation**
  - Avoid data leakage across splits

---

### 3.4 Data Quality Checks

- Verify:
  - Safe/unsafe **class balance**
  - Coverage across:
    - Easy scenes
    - Hard/boundary scenes
  - Remove:
    - Invalid samples
    - Duplicate entries

---

## 4. Barrier Model Design

- Model: **Neural Barrier Function**

$$B_\theta(z, o)$$

- Architecture:
  - **MLP**
- Input:

$$[z, o]$$

- Output:
  - Scalar barrier value

---

## 5. Training Strategy

### 5.1 Loss Function

Combined objective:

- **Safe sign loss**
- **Unsafe sign loss**
- **Transition CBF constraint loss**

---

### 5.2 Transition Constraint

$$B(z_{k+1}^{nom}, o) \geq (1 - \alpha\Delta)\, B(z_k, o)$$

- Enforces **forward invariance of safety**

---

### 5.3 Dual-Batch Training

Each iteration:

- Batch 1 $\rightarrow$ State-label dataset
- Batch 2 $\rightarrow$ Transition dataset

Optimize:

- Weighted sum of all loss terms

---

### 5.4 Optimization

- Optimizer: **Adam**
- Tune:
  - $\lambda_s, \lambda_u, \lambda_d$
  - $\alpha, \Delta$

---

## 6. Training Monitoring & Model Selection

### 6.1 Metrics to Track

- Safe classification accuracy
- Unsafe classification accuracy
- Transition violation rate
- Total weighted loss trend

---

### 6.2 Checkpoint Selection

- Do **not rely only on loss**
- Select based on:
  - **CBF constraint satisfaction**
  - Stability during rollout

---

## 7. Runtime Integration (CBF2 Planning)

### Pipeline

1. Compute nominal update:

$$z_{k+1}^{nom}$$

   (Goal + Prior planner)

2. Compute gradient:

$$d = \nabla_z B_\theta(z_{k+1}^{nom}, o)$$

3. Compute CBF correction (CBF2 step)
4. Apply correction:

$$z_{k+1}^{safe}$$

5. Continue:

$$z \leftarrow z_{k+1}^{safe}$$

---

## 8. Planning & Evaluation Pipeline

- Integrate CBF into:
  - Evaluation loop
  - Simulation loop
- Keep baseline pipeline unchanged for comparison

---

## 9. Final Evaluation Metrics

**Task Performance**

- Goal-reaching rate
- Collision-free rate
- True success rate

---

**Efficiency Metrics**

- Planning time
- Path length

---

**CBF-Specific Metrics**

- Average correction magnitude
- Reduction in safety violations
- Frequency of CBF interventions

---

## 10. Ablation Studies (Mandatory)

Evaluate impact of each component:

- Without transition loss
- Without CBF correction at inference
- Sensitivity to:
  - $\alpha$
  - $\Delta$

---

## 11. Reproducibility Package

Include:

- Fixed configs
- Fixed seeds
- Saved datasets
- Best model checkpoint
- Scripts for:
  - Data generation
  - Training
  - Evaluation
  - Simulation
  - and etc...

---

**12. Do NOT modify existing codebase**

**Create new scripts for:**

1. **CBF Data Generation**
2. **CBF Training**
3. **CBF Planning & Evaluation & Simulation**

---